

INF-3200: DISTRIBUTED SYSTEMS FUNDAMENTALS

ASSIGNMENT 2: DYNAMIC MEMBERSHIP

Sera Madeleine Elstad and Victor Zimmer

October 25, 2024

I. INTRODUCTION

The aim of this assignment is to implement dynamic membership for a distributed key-value store using the Chord protocol. Specifically, the objective is to enable nodes to join and leave the decentralized network dynamically while ensuring stable data access and efficient routing. This project builds on the previous implementation, extending its functionality to evaluate the system's scalability, robustness, and tolerance to node failures. By allowing dynamic membership, the goal is to demonstrate how the network maintains stability and consistent performance under fluctuating conditions.

II. DESIGN AND IMPLEMENTATION

A. Node info

When the node information endpoint is accessed, it provides an overview of the node's current status in the Chord ring. This includes the node's unique identifier, its position in the ring, its successor, and other relevant nodes, such as its predecessor and finger table entries. The data is then returned in a structured JSON format.

B. Dynamic membership

1) *Dynamic joining*: When a new node joins the Chord ring, it connects to an existing node to request information about the network. The new node uses this information to determine its correct position and updates its own successor and predecessor pointers accordingly.

2) *Dynamic leaving*: When a node wants to leave the Chord ring, it updates its predecessor and successor nodes to bypass itself. The node retrieves its current predecessor and successor, then sends an HTTP request to its predecessor to update its successor pointer to the node's current successor. Similarly, it instructs its successor to update its predecessor pointer to the node's current predecessor.

C. Crash and Recovery

1) *Crash*: The endpoint for simulating a node crash first checks if the node is already marked as crashed. If it is, an error response is returned, indicating that the node is

unavailable. If the node is still operational, its status is updated to "crashed", separating it from the network.

Once in this state, the node will no longer respond to any incoming requests, and any attempt to interact with it via other routes will result in failure, showing that the node is offline. This confirms that the node is no longer active or participating in network activities.

2) *Recovery*: When a node wants to recover it first resets the node's status, indicating that it is operational again. Once the node is marked as "recovered," it attempts to reconnect to the network by reaching out to its neighboring nodes. The node checks if its predecessor or successor is online and accessible. If one of these nodes is available, it uses them to re-establish its position in the ring.

If the immediate neighbors are not accessible, the node may use entries from its finger table to locate another active node in the network. This process ensures that the node can smoothly reintegrate and resume its role in routing and storage operations. Once reconnected, the node is fully functional. If the node is unable to connect to any of its neighbours the reconnection will fail.

3) *Maintaining Network Consistency*: The system employs a heartbeat mechanism to monitor the status of the successor of a node, ensuring the Chord ring remains stable. Every 5 seconds, a background thread sends an HTTP request to the successor of the node to check if it is responsive. If the successor responds, the node continues its operation as normal, storing the latest information about the successor. However, if the successor fails to respond (indicating that it might be down), the node initiates a recovery procedure.

In the event of an unresponsive successor, the system first logs the failure and attempts to replace the dead successor using previously received information. If a backup successor (which is the successor of the successor that now is unresponsive) is available, the node updates its configuration to use this node as the new successor, helping to restore connectivity and maintain the network's functionality. If no backup information is available, the node waits for the next interval to retry and find a viable replacement.

III. EXPERIMENTS

A. Network growth and shrinkage experiment:

The experiment performs 1000 iterations of join and leave tests across network sizes of 2, 4, 8, 16, and 32 nodes. A network of 32 nodes is initialized, starting with two connected nodes (where the first node is nprime). The network size is then doubled incrementally: from 4 to 8, 8 to 16, and finally 16 to 32 nodes. Once the network reaches 32 nodes, the leave operation begins, shrinking the network size at each step from 32 to 16, 16 to 8, and so on until only 2 nodes remain. This sequence is repeated 1000 times to measure average times, with standard deviations showing the variability, as illustrated in figure 1.

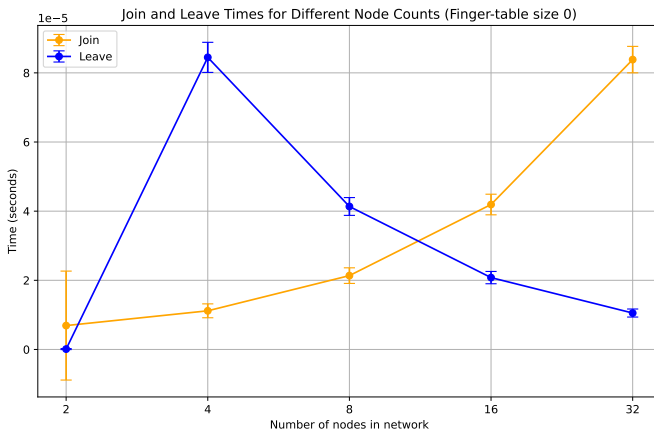


Fig. 1: 1000 iterations of Join and leave without finger table

As shown in figure 1, join times increase with network size, indicating more overhead and complexity as more nodes are added. The standard deviation also increases for larger networks, reflecting the variability in stabilization times.

However, leave times indicate that smaller networks take longer to stabilize, which is surprising. This could be because when nodes become isolated or part of smaller groups, they need more time to reset and stabilize themselves before completely disconnecting. This tiny delay has a considerable effect when the overall leave time is short, which may explain why leave times increase with smaller network sizes.

B. Crash Tolerance Experiment

IV. DISCUSSION

A. Hashed vs generated id

In the previous report, the benefits of **generated IDs** were highlighted, particularly for ensuring an evenly distributed ring

and preventing node clustering, especially in smaller networks.

While **hashed IDs** allow crashed nodes to rejoin in the same position, **generated IDs** remain a good choice for this assignment. Chord focuses on overall network stability and efficiency, rather than the exact rejoin position of individual nodes. When a node crashes and rejoins at a new position, the network adjusts automatically, with neighboring nodes updating successor and predecessor pointers. It's finger table is cleared and will have to be recalculated. This ensures smooth recovery, even if the node rejoins in a different position.

In addition, generated IDs ensure that node distribution remains balanced as the network grows, preventing unequal key ranges and reducing the need for manual balancing. This makes the system scalable and capable of handling dynamic joins and leaves.

B. Burst of crashed nodes

Our current approach can handle any number of crashed nodes as long as they're not successive. In the case of successive node failures a strategy to find the next successor by working backwards from the predecessor or a finger table entry would be needed. In our implementation this would likely be implemented by a new endpoint for a node to request the last online predecessor, with an input of the requesting node. This request would then travel backwards until a predecessor is not online and the local node would be returned to the requesting node. This does come with the possibility of excluding live nodes in the edge case where two successive nodes fail in to locations, with one or more live nodes between them. This would have to be handled, which is possible as these nodes will be aware that their successor is gone and unable to find a predecessor. Likely they would need to rejoin the network thru a node in the finger table.

V. CONCLUSION

Dynamic joining and leaving work as it should, with simulated crashing and recovery. However, directly successive node failures makes the network unable to stabilize.

REFERENCES